

Corso di Laurea in Informatica A.A. 2024-2025

Laboratorio di Sistemi Operativi

Alessandra Rossi

Esercizio 1

@ Scrivere due programmi C, **server.c** e **client.c**, che comunicano tramite socket. Il **server.c** crea una socket locale, ed ogni volta che instaura una connessione con un client, mediante **fork** crea un nuovo processo. Tale processo, dovrà leggere sul socket il messaggio scritto dal client e scrivere il messaggio che il server manda al client. Il **client.c** dovrà connettersi al socket locale su cui scriverà il messaggio da mandare al server.

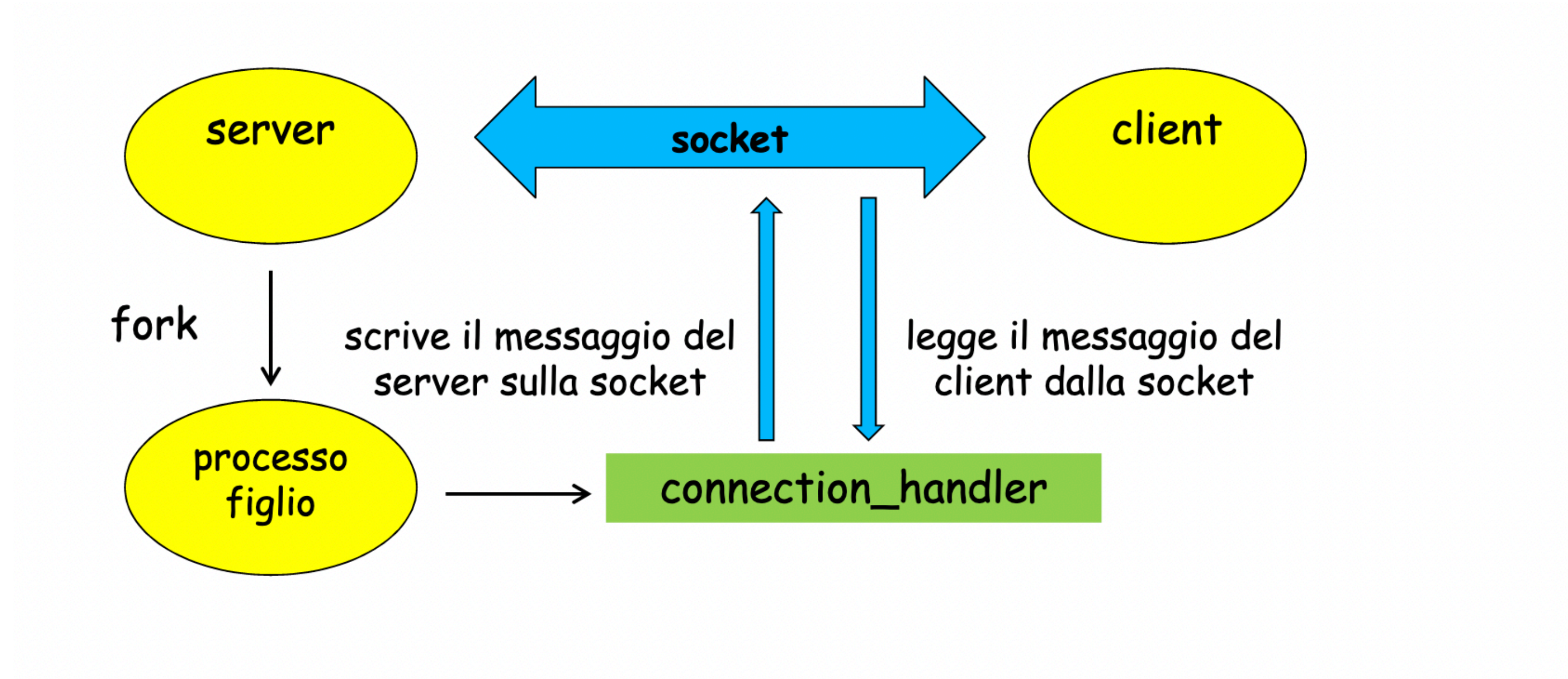
(lanciare l' esecuzione dei due programmi su due shell distinti della stessa macchina)

Esecuzione

```
$ ./server (shell1) MESSAGGIO DA CLIENT: Saluti da client
```

```
$ ./client (shell2) MESSAGGIO DA SERVER: Saluti dal server
```


Esercizio 1



Esercizio 2

- Ad ogni nuova connessione, il server scrive sul socket l'ora corrente, poi chiude la connessione e si rimette in attesa di nuove connessioni
- Implementare anche un client che riceve l'ora dal server e la stampa sul terminale
- Provare a lanciare diversi client in rapida successione

Esercizio 2

- Implementare un server che fornisce ai client l'ora esatta, usando un socket locale
 - sugg.: una stringa che rappresenta l'ora esatta si può ottenere così:

```
#include <time.h>
...
char buffer[26];
time_t ora;
time(&ora);
printf(" Ora esatta : %s\n", ctime_r(&ora, buffer));
```

- la funzione `time` restituisce l'ora in un formato interno (`time_t`)
- la funzione `ctime_r` trasforma il formato interno in stringa; ha bisogno di un buffer di (almeno) 26 caratteri

Esercizio 3

@ Scrivere due programmi C, **chef.c** (server) e **cook.c** (client). Il server crea un socket chiamato "**ricetta**" e tramite essa fornisce una ricetta a tutti i client che la richiedono. La ricetta è formata da una sequenza di stringhe terminate dal carattere '\0'. Il client si connette al socket chiamato "**ricetta**" e legge la ricetta fornita dal server. Man mano che il client legge la ricetta la mostra sullo standard output e quindi termina. Il server, crea un processo figlio che evade la richiesta del client. (lanciare i due programmi, su due shell distinte della stessa macchina)

Esecuzione:

```
$ ./chef.out (server) (shell1)  
$ ./cook.out (client) (shell2)
```

Output:

lato server: La ricetta è stata scritta nel socket

lato client: prova, prova, prova, prova,
 prova, & prova.

Esercizio 1: Server

```
#include <stdio.h>
#include <sys/socket.h>
#include <sys/un.h>
#include <sys/types.h>
#include <unistd.h>
int connection_handler(int connection_fd)
{
    int nbytes;
    char buffer[256];
    //legge da socket il mess. del client
    nbytes = read(connection_fd, buffer, 256);
    buffer[nbytes] = 0;
    printf("MESSAGGIO DA CLIENT: %s\n", buffer);
    nbytes = sprintf(buffer, "Saluti dal server");
    //Scrive su socket il mess. del server
    write(connection_fd, buffer, nbytes);
    close(connection_fd);
    return 0;
}
```

Funzione chiamata dal processo figlio per comunicare con il client tramite socket

sockfd

Esercizio 1: Server

```
int main(void)    //server
{
    struct sockaddr_un address;//struttura degli indirizzi
    int socket_fd, conn_fd;
    size_t address_length;    //dichiarazione delle variabili
    pid_t child;
    socket_fd = socket(PF_LOCAL, SOCK_STREAM, 0); //crea socket
    if(socket_fd < 0)
    {
        printf("socket() failed\n");
        return 1;
    }
    unlink("./demo_socket"); /*Rimuove la socket se già esiste */
    //valorizzo i campi della struttura sockaddr_un
    address.sun_family = AF_LOCAL; //setta il tipo di dominio
    address_length = sizeof(address.sun_family) +
                    sprintf(address.sun_path, "./demo_socket");
```


Esercizio 1: Server

```
if(bind(socket_fd, (struct sockaddr *) &address,
address_length) != 0)
{printf("bind() failed\n");
return 1;}
if(listen(socket_fd, 5) != 0)//si mette in ascolto
{printf("listen() failed\n");
return 1;}
//crea la connessione con i client
while((conn_fd = accept(socket_fd, (struct sockaddr *) &address,
&address_length)) > -1)
{
child = fork();//crea il figlio
if(child == 0){
return connection_handler(conn_fd);//comunica con client
}
close(conn_fd);
} //end while
close(socket_fd);
unlink("./demo_socket"); /*Rimuove la socket*/
return 0;
} //end server
```


Esercizio 1: Client

```
#include <stdio.h>
#include <sys/socket.h>
#include <sys/un.h>
#include <unistd.h>
int main(void) //client
{
    struct sockaddr_un address;
    int socket_fd, nbytes;
    size_t address_length;
    char buffer[256];
    socket_fd = socket(PF_LOCAL, SOCK_STREAM, 0); //crea socket
    if(socket_fd < 0)
    {printf("socket() failed\n");
      return 1;}
    //valorizza i campi della struttura indirizzi
    address.sun_family = AF_LOCAL;
    address_length = sizeof(address.sun_family) +
                    sprintf(address.sun_path, "./demo_socket");
}
```

} dichiarazione delle variabili

Esercizio 1: Client

```
//si propone di connettersi al socket del server prima accept
if(connect(socket_fd, (struct sockaddr *) &address,
address_length) != 0)
{
    printf("connect() failed\n");
    return 1;
}

nbytes = sprintf(buffer, "Saluti da client");
//scrive su socket il mess. da mandare al server
write(socket_fd, buffer, nbytes);
//legge da socket il mess. del server
nbytes = read(socket_fd, buffer, 256);
buffer[nbytes] = 0;

printf("MESSAGGIO DA SERVER: %s\n", buffer);

close(socket_fd);

return 0;
} //end client
```

Esercizio 2: Server

```
#define SOCKET_NAME "/tmp/my_first_socket"

static int gestisci(int);

int main(int argc, char **argv)
{
    int listen_sd, connect_sd; // Socket descriptor

    struct sockaddr_un my_addr, client_addr;
    socklen_t client_len;

    my_addr.sun_family = AF_LOCAL;
    strcpy(my_addr.sun_path, SOCKET_NAME);

    // Server section: create a local socket
    if ( (listen_sd = socket(PF_LOCAL, SOCK_STREAM, 0)) < 0)
        perror("socket"), exit(1);
```


Esercizio 2

```
// remove socket file if present
unlink(SOCKET_NAME);

// bind socket to pathname
if ( bind(listen_sd, (struct sockaddr *) &my_addr, sizeof(my_addr)) < 0)
    perror("bind"), exit(1);

// put socket in listen state
if ( listen(listen_sd, 1) < 0)
    perror("listen"), exit(1);

while (1) {
    client_len = sizeof(client_addr);
    fprintf(stderr, " sizeof(client_addr)=%d \n", client_len);

    if ( (connect_sd = accept(listen_sd, (struct sockaddr *) &client_addr, &client_len)) < 0)
        perror("accept"), exit(1);

    fprintf(stderr, " new connection \n");
    fprintf(stderr, " client address: %100s\n ", client_addr.sun_path);

    // handle the connection
    gestisci(connect_sd);
    close(connect_sd);
}

return 0;
```

Esercizio 2

```
int gestisci(int sd)
{
    char buf[100];
    int n;

    time_t ora;
    time(&ora);
    printf(" Ora: %s\n", ctime_r(&ora, buf));
    printf(" Ora: %d\n", strlen(buf));
    write(sd, buf, 26);
    return 0;
}
```


Esercizio 2: client

```
#define SOCKET_NAME "/tmp/my_first_socket"
```

```
static int client(void);
```

```
int main(int argc, char **argv)
{
    return client();
}
```

```
// Client section: Sends integers from 0 to 4, at 1 second intervals, and then terminates
```

```
int client(void)
```

```
{
    char msg[100];
    struct sockaddr_un srv_addr;
    int sd, i;
    ssize_t temp; /* signed size_t */
```

```
    signal(SIGPIPE, SIG_IGN);
```

```
    // crea il socket
```

```
    sd = socket(PF_LOCAL, SOCK_STREAM, 0);
```

```
    if (sd < 0) perror("socket"), exit(1);
```

```
    srv_addr.sun_family = AF_LOCAL; // unsigned short int
    strcpy(srv_addr.sun_path, SOCKET_NAME);
```

Esercizio 2

```
// si connette all'indirizzo predefinito
if ( connect(sd,(struct sockaddr *) &srv_addr, sizeof(srv_addr) ) < 0)
    perror("connect"), exit(1);

sleep(1);
read(sd, msg, 26);
printf(" client riceve: *%s*\n", msg);
printf(" client riceve: %d\n", strlen(msg));
return 0;

}
```


Esercizio 3: server

```
#include <stdio.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <sys/un.h> /* For AF_UNIX sockets */

int writeRicette (int fd) {

    static char* line1 = "prova, prova, prova, prova,";
    static char* line2 = "prova, & prova.";

    write (fd, line1, strlen (line1) + 1); /* Write first line */
    write (fd, line2, strlen (line2) + 1); /* Write second line */

}
```

chiamata dal server per scrivere la ricetta nel socket

Esercizio 3: server

```
int main (void) {

    int serverFd, clientFd, serverLen, clientLen;
    struct sockaddr_un serverUNIXAddress; /* Server address */
    struct sockaddr_un clientUNIXAddress; /* Client address */

    struct sockaddr* serverSockAddrPtr; /* Ptr to server address */
    struct sockaddr* clientSockAddrPtr; /* Ptr to client address */

    //Parametri da passare alla bind
    serverSockAddrPtr = (struct sockaddr*) &serverUNIXAddress;
    serverLen = sizeof (serverUNIXAddress);

    //Parametri da passare alla accept
    clientSockAddrPtr = (struct sockaddr*) &clientUNIXAddress;
    clientLen = sizeof (clientUNIXAddress);
```


Esercizio 3: server

```
serverFd = socket (PF_LOCAL, SOCK_STREAM, 0);
serverUNIXAddress.sun_family = AF_LOCAL; /* Set domain type */
strcpy (serverUNIXAddress.sun_path, "ricetta"); /* Set name */

unlink ("ricetta"); /* Remove file if it already exists */
bind (serverFd, serverSockAddrPtr, serverLen); /* Create file */
listen (serverFd, 5); /* Maximum pending connection length */

while (1) {
    clientFd = accept (serverFd, clientSockAddrPtr, &clientLen);
    if (fork () == 0) { /* Create child to send receipe */
        writeRicette (clientFd); /* Send the recipe */
        printf("La ricetta è stata scritta nel socket\n");
        close (clientFd); /* Close the socket */
        exit (0); /* Terminate */
    } else
        close (clientFd); /* Close the client descriptor */
}
} //END Chef-server
```


Esercizio 3: client

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <sys/un.h> /* For AF_UNIX sockets */

int readRicetta (int fd) {
    char str[200];
    while (readLine (fd, str)) /* Read lines until end-of-input */
        printf ("%s\n", str); /* Echo line from socket */
}

int readLine (int fd, char *str) {
    int n;
    do { /* Read characters until '\0' or end-of-input */
        n = read (fd, str, 1); /* Read one character */
    } while (n > 0 && *str++ != '\0');
    return (n > 0); /* Return false if end-of-input */
}
```

chiamata dal client per leggere la ricetta nel socket

Esercizio 3: client

```
int main (void) {

    int clientFd, serverLen, result;
    struct sockaddr_un serverUNIXAddress;
    struct sockaddr* serverSockAddrPtr;

    serverSockAddrPtr = (struct sockaddr*) &serverUNIXAddress;
    serverLen = sizeof (serverUNIXAddress);

    clientFd = socket (PF_LOCAL, SOCK_STREAM, 0);
    serverUNIXAddress.sun_family = AF_LOCAL;
    strcpy (serverUNIXAddress.sun_path, "ricetta");

    /*In attesa di connettersi alla socket*/
    do {
        result = connect (clientFd, serverSockAddrPtr, serverLen);
        if (result == -1) sleep (1); /* Wait and then try again */
    } while (result == -1);

    readRicetta (clientFd); /* Read the recipe */
    close (clientFd); /* Close the socket */
    exit (0); /* Done */
} //end cook-client
```




Università degli Studi di Napoli Federico II

THANK YOU FOR YOUR ATTENTION

TRUST ME ...
I AM A ROBOT!

